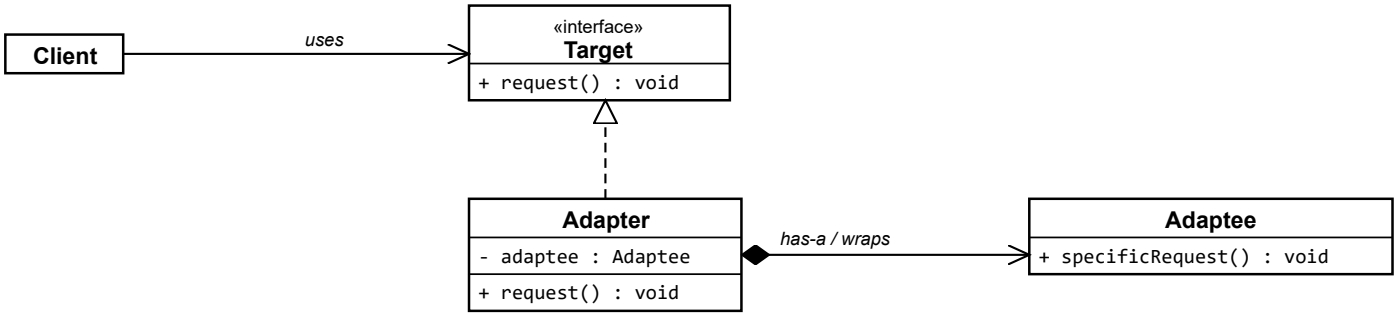
04 Adapter

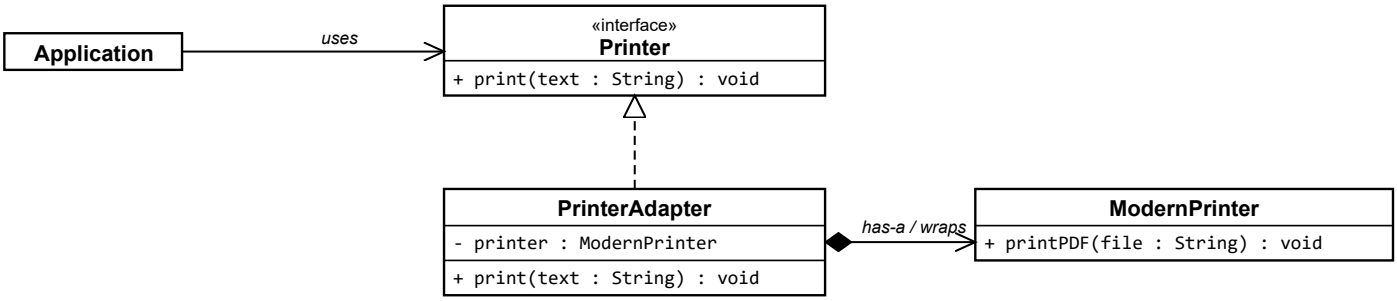
Structural · “wrapper”

Intent (GoF)	**Convert the interface of a class into another interface clients expect.** Adapter lets classes work together that couldn’t otherwise because of incompatible interfaces.
Use when	An **existing / legacy / third-party / vendor class** has the functionality you need but the **wrong method names or formats**, and you **cannot or should not change** either side. Real life: USB-C→HDMI, travel plug, card reader.
Trigger words	“incompatible interface”, “legacy”, “supplied by an external vendor, can’t modify”, “new device/gateway provides a different method”, “hundreds of calls to the old method”, “third-party web services with different method signatures”, “developed by different vendors / technologies”.
Participants	**Target** (interface the client expects) · **Adapter** (implements Target, holds Adaptee, translates) · **Adaptee** (existing incompatible class) · **Client** (uses Target only).
Characteristics	Client and adaptee interfaces are incompatible · adapter **wraps** the adaptee, hiding it · client calls the adapter via the target interface · adapter **translates** the request into what the adaptee expects. Only the adapter knows the adaptee.

UML — generic (object adapter)



UML + Java — Printer (lecture): old app calls `print()`, new printer has `printPDF()`



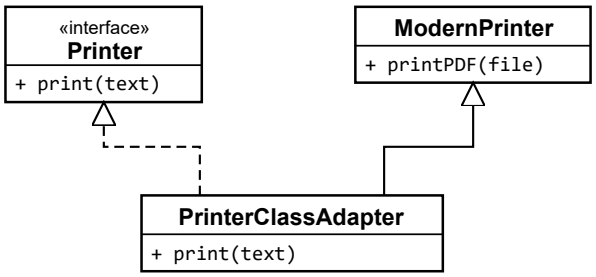

```
// 1. TARGET - what the application expects
public interface Printer {
    void print(String text);
}
// 2. ADAPTEE - existing class, cannot change
public class ModernPrinter {
    public void printPDF(String file) {
        System.out.println("Printing PDF : " + file);
    }
}
```

```
// 3. ADAPTER - implements Target, wraps Adaptee
public class PrinterAdapter implements Printer {
    private ModernPrinter printer;                  // has-a
    public PrinterAdapter(ModernPrinter printer) { this.printer
= printer; }
    @Override
    public void print(String text) {
        printer.printPDF(text);                     // translate
call
    }
}
// 4. CLIENT - unchanged, knows only Printer
Printer p = new PrinterAdapter(new ModernPrinter());
p.print("Report.pdf");      // Output: Printing PDF : Report.pdf
```

Flow: client calls `print()` on Printer → goes to PrinterAdapter → adapter calls `ModernPrinter.printPDF()` → adaptee does the work. Without the adapter: compile error “cannot find symbol printDocument()”. Payment example: `StripeAdapter implements Payment { pay(amt) → stripe.makePayment(amt) }`. Java library adapters: `InputStreamReader` (InputStream→Reader), `Arrays.asList()` (array→List), `Collections.enumeration()`.

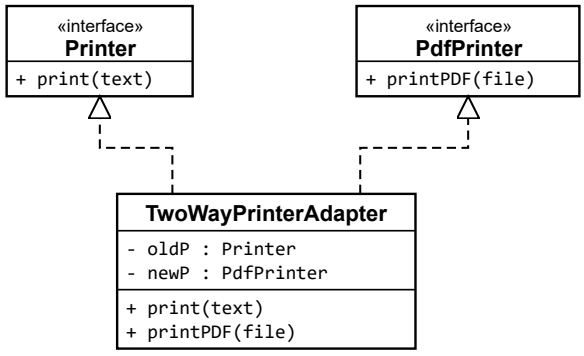
Object adapter vs class adapter vs two-way adapter

	Object adapter (default, used above)	Class adapter	Two-way adapter
How	**Composition**: implements Target, **holds** an Adaptee object	**Inheritance**: implements Target **and extends** Adaptee	Implements **both** interfaces (old and new) and translates in both directions
Java	`class A implements Target { Adaptee x; }`	`class A extends Adaptee implements Target`	`class A implements OldIf, NewIf`
Pros	Works with the adaptee **and all its subclasses**; adaptee can be swapped at runtime; loose coupling (composition over inheritance)	No extra object / delegation call; can **override** adaptee behaviour	One object usable by **old and new clients** at the same time (gradual migration, system integration)
Cons	One more object + delegation call	Java has single class inheritance, so it can't extend two classes; fails if adaptee is `final`; tied to one adaptee class (tight coupling)	More complex; must keep both translations correct; bigger class
Use when	Almost always; **default exam answer**	Target is an interface, adaptee not final, only that one class needs adapting	Both systems must keep calling each other during a migration (old app ↔ new printer / old bank API ↔ new loan module)



is-a Target AND is-a Adaptee (no field)

```
public class PrinterClassAdapter extends ModernPrinter
implements Printer {
    public void print(String text) { printPDF(text); }   //
inherited method
}
Printer p = new PrinterClassAdapter();
p.print("Report.pdf");
```



implements BOTH interfaces: old and new clients can use it

```
public interface PdfPrinter { void printPDF(String file); }
public class TwoWayPrinterAdapter implements Printer, PdfPrinter
{
    private Printer oldP; private PdfPrinter newP;
    public TwoWayPrinterAdapter(Printer o, PdfPrinter n) { oldP
= o; newP = n; }
    public void print(String text)    { newP.printPDF(text); }
// old client -> new
    public void printPDF(String file) { oldP.print(file); }
// new client -> old
}
```

Advantages / disadvantages / challenge → mechanism

Advantages	Disadvantages	Challenge → mechanism
Reuse legacy / third-party code without modification	More classes (one adapter per adaptee)	Many vendors → one adapter per vendor behind a common Target; create them with a **Factory**
Loose coupling: client depends on Target only	Extra layer of indirection	Data/format translation (XML/JSON, units, cents) → mapping/converter inside the adapter; unit-test each adapter
Open/Closed: new adapters without changing client	Confusing if overused	Vendor API changes → only that adapter changes; version the adapters
Easy integration & testing (mock the adaptee)	Small performance overhead	Features missing in adaptee → adapter implements default/emulated behaviour or throws a clear exception

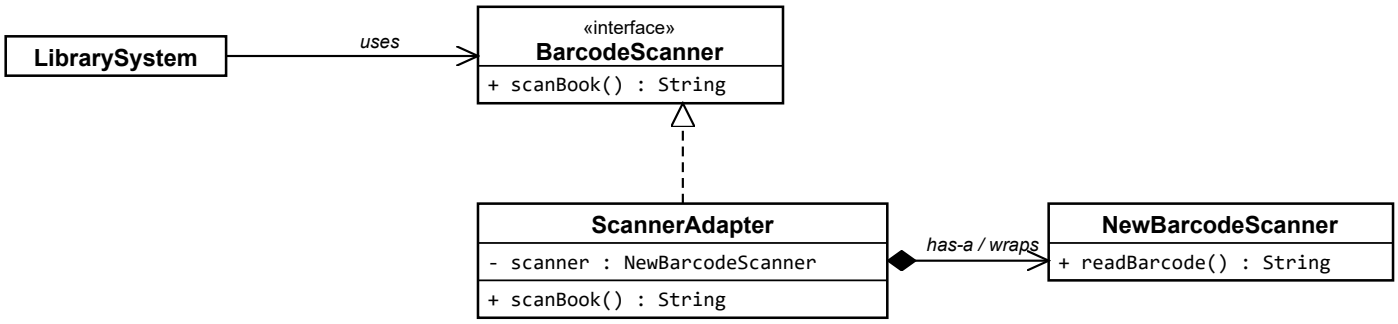
"Adapter needs more code. Agree?" Yes, extra classes — but the payoff is large for legacy systems that can't change but are stable. **Why not change the interfaces / the client?** Adaptee is third-party/vendor/compiled (no source), changing tested code risks bugs, many clients depend on the old interface, licence/contract forbids it.

Questions & model answers

In this section: **Library barcode scanner** ADAPTER · **Parking legacy payment** ADAPTER · **Travel third-party APIs** ADAPTER · **Restaurant event sites** ADAPTER · **Adapter / Facade / Proxy** COMPARE

ADAPTER **LIBRARY BARCODE SCANNER** — Library: system calls `scanBook()` hundreds of times; new scanner provides `readBarcode()`. Reason not to modify existing code? Pattern and why? Identify participants. Draw UML. *Quiz 3 Q1–Q4*

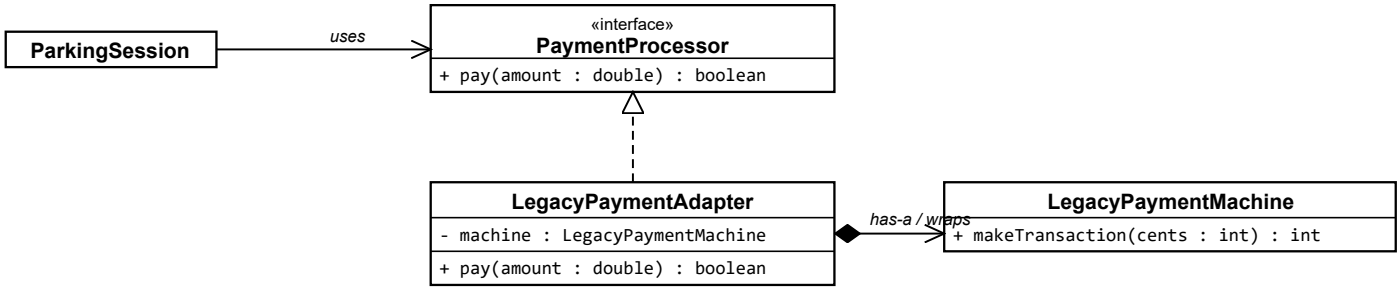
Not modifying: hundreds of call sites ⇒ huge change, high risk of new bugs, full re-testing, downtime; tested working code should stay closed for modification (OCP); the scanner class comes from a vendor so it can't be changed either. **Pattern: Adapter** — the two interfaces are incompatible but do the same job; a `ScannerAdapter` implements the old `scanBook()` and internally calls `readBarcode()`, so the library code is untouched and the new scanner is reused. **Participants:** Client = `LibrarySystem`; Target = `BarcodeScanner` (`scanBook()`); Adapter = `ScannerAdapter`; Adaptee = `NewBarcodeScanner` (`readBarcode()`).




```
public interface BarcodeScanner { String scanBook(); }                 // Target (old)
public class NewBarcodeScanner { public String readBarcode() { return "978-0134685991"; } }  // Adaptee
public class ScannerAdapter implements BarcodeScanner {                 // Adapter
    private final NewBarcodeScanner scanner;
    public ScannerAdapter(NewBarcodeScanner s) { this.scanner = s; }
    public String scanBook() { return scanner.readBarcode(); }
}
BarcodeScanner sc = new ScannerAdapter(new NewBarcodeScanner());   // only this line changes
String isbn = sc.scanBook();                                          // all old calls unchanged
```

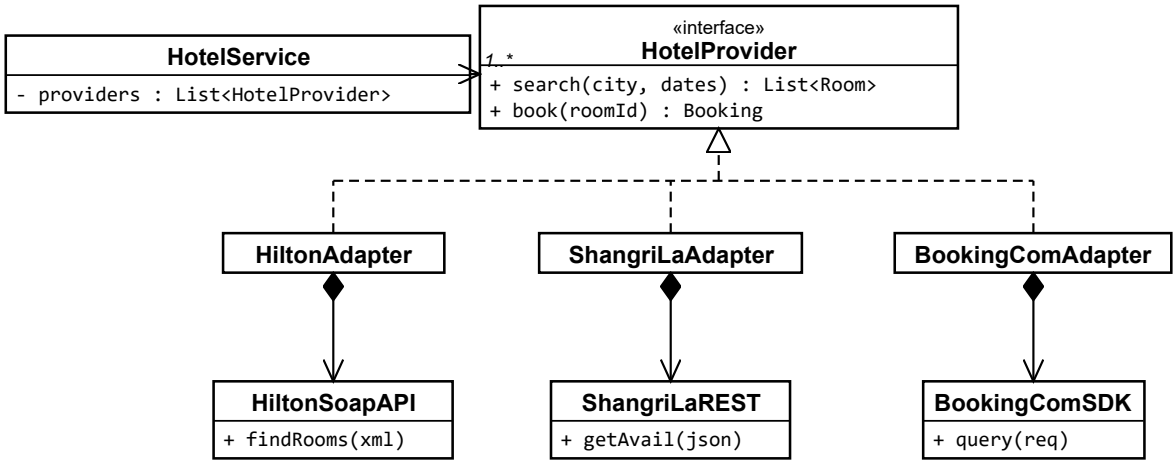
ADAPTER **PARKING LEGACY PAYMENT** — Parking: integrate a legacy payment machine supplied by an external vendor; can't modify it. *Quiz 1 Q5*

Adapter. Define the interface the parking system expects (`PaymentProcessor.pay(amount)`); write `LegacyPaymentAdapter` that implements it, holds the vendor's `LegacyPaymentMachine` and converts the call (e.g. rupees → cents, result code → boolean). `PaymentDueState` calls `processor.pay()` and never sees the legacy class.




```
public class LegacyPaymentAdapter implements PaymentProcessor {
    private final LegacyPaymentMachine machine = new LegacyPaymentMachine();
    public boolean pay(double amount) {
        int code = machine.makeTransaction((int) Math.round(amount * 100));   // convert units
        return code == 0;                                                      // convert result
    }
}
```

ADAPTER **TRAVEL THIRD-PARTY APIS** — Travel booking: which pattern to connect each sub-system with different third-party web services? Challenges and mechanism. *2022 Q1.3 (10)*



Adapter (one per provider): HotelService talks to a common `HotelProvider` interface; `HiltonAdapter`, `ShangriLaAdapter`… translate to each provider's SOAP/REST/SDK signatures. Same for airlines and taxis. **Challenges:** many adapters to write/maintain; different data formats and error codes; provider API version changes; slow/unavailable services. **Mechanisms:** AdapterFactory/registry to pick adapters by config; a common DTO + mapper in each adapter; isolate version changes to one adapter; timeouts, retries, caching and fallback to another provider.

ADAPTER **RESTAURANT EVENT SITES** — Restaurant must show events from websites built by different vendors with different technologies — how to integrate? *2023 Q3.3 (5)*

Same idea: `EventSource` target interface (`getEvents()`); one adapter per website (`RssEventAdapter`, `JsonApiEventAdapter`, `HtmlScraperAdapter`) converting each source into a common `Event` object. The restaurant site iterates a list of `EventSource`; a new vendor = a new adapter. (A Facade can then combine all sources into one `EventFeed`.)

COMPARE **ADAPTER / FACADE / PROXY** — Adapter vs Facade vs Proxy

	Adapter	**Facade**	**Proxy**
Goal	**Convert** interface	**Simplify** a subsystem	**Control access**
Interface	different from adaptee's	new, higher-level	**same** as real subject
Wraps	usually one object	many objects	one object
Memory tip	plug adapter	building front / waiter	security guard

Mistakes: Adapter *extends* Target interface wrongly or client calls the adaptee directly · drawing association from adaptee to adapter (arrow goes adapter → adaptee) · modifying the adaptee · confusing with Facade.